

# TRABAJO PRÁCTICO 5: Optimización y Procesamiento de Datos

Ignacio Figueroa – Comisión 7

Base de Datos II

## ACTIVIDAD 1: Optimización mediante Índices

### 1) Respuestas Teóricas

a) Identificar qué campo requiere un Índice Simple y proponer el comando para crearlo. El campo que requiere un índice simple es `numero_seguimiento`, ya que es un identificador único por el cual se realizan búsquedas frecuentes. Al ser único, podemos agregar la propiedad `unique: true` para asegurar la integridad de los datos.

```
db.envios.createIndex({ numero_seguimiento: 1 }, { unique: true })
```

b) Identificar qué campos requieren un Índice Compuesto para optimizar las búsquedas por estado y ciudad. Proponer el comando. Para optimizar las consultas que filtran por estado y ciudad\_destino simultáneamente, se requiere un índice compuesto que incluya ambos campos. El orden de los factores importa según la selectividad; asumiremos el siguiente orden estándar:

```
db.envios.createIndex({ estado: 1, ciudad_destino: 1 })
```

c) Explicar qué sucede internamente en MongoDB cuando se realiza una búsqueda en una colección de gran volumen que no posee índices. Cuando no existen índices, MongoDB se ve obligado a realizar un COLLSCAN (Collection Scan). Esto significa que el motor de la base de datos debe leer, uno por uno, todos los documentos de la colección en el disco para verificar si cumplen con las condiciones del filtro. En colecciones de gran volumen, esto causa un alto consumo de CPU, lecturas excesivas de E/S en disco y un incremento drástico en el tiempo de respuesta de la consulta.

## 2) Implementación Práctica

```
mongo
> use logistica_db
switched to db logistica_db
> db.envios.insertMany([
  { numero_seguimiento: "TRK-001", estado: "En Camino", ciudad_destino: "Buenos Aires", peso: 5.5 },
  { numero_seguimiento: "TRK-002", estado: "Entregado", ciudad_destino: "Rosario", peso: 25.0 },
  { numero_seguimiento: "TRK-003", estado: "Entregado", ciudad_destino: "Buenos Aires", peso: 12.2 },
  { numero_seguimiento: "TRK-004", estado: "Retenido", ciudad_destino: "Córdoba", peso: 42.1 },
  { numero_seguimiento: "TRK-005", estado: "En Camino", ciudad_destino: "Mendoza", peso: 8.0 },
  { numero_seguimiento: "TRK-006", estado: "Entregado", ciudad_destino: "Rosario", peso: 3.2 },
  { numero_seguimiento: "TRK-007", estado: "En Camino", ciudad_destino: "Córdoba", peso: 18.5 },
  { numero_seguimiento: "TRK-008", estado: "Entregado", ciudad_destino: "Buenos Aires", peso: 31.0 },
  { numero_seguimiento: "TRK-009", estado: "Retenido", ciudad_destino: "Mendoza", peso: 1.5 },
  { numero_seguimiento: "TRK-010", estado: "Entregado", ciudad_destino: "Buenos Aires", peso: 21.5 }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a3a9dc388148ffafe262cb4'),
    '1': ObjectId('6a3a9dc388148ffafe262cb7'),
    '2': ObjectId('6a3a9dc388148ffafe262cb8'),
    '3': ObjectId('6a3a9dc388148ffafe262cb9'),
    '4': ObjectId('6a3a9dc388148ffafe262cba'),
    '5': ObjectId('6a3a9dc388148ffafe262cbb'),
    '6': ObjectId('6a3a9dc388148ffafe262cbc'),
    '7': ObjectId('6a3a9dc388148ffafe262cbd'),
    '8': ObjectId('6a3a9dc388148ffafe262cbe'),
    '9': ObjectId('6a3a9dc388148ffafe262cbf')
  }
}
> db.envios.createIndex({ numero_seguimiento: 1 }, { unique: true })
db.envios.createIndex({ estado: 1, ciudad_destino: 1 })
< estado_1_ciudad_destino_1
Atlas atlas-u7v6mb-shard-0 [primary] logistica_db
```

## ACTIVIDAD 2: Diagnóstico y Evaluación de Rendimiento

a) Mencionar qué comando de MongoDB permite visualizar si una consulta está realizando un escaneo completo de la colección (COLLSCAN) o utilizando un índice (IXSCAN).

Se utiliza el método `.explain()` asociado a la consulta, pasando el parámetro "executionStats" para obtener el detalle de la ejecución. Por ejemplo:  
`db.coleccion.find().explain("executionStats").`

b) ¿Qué significan los campos `executionTimeMillis` y `totalDocsExamined`?

- **executionTimeMillis:** Indica el tiempo total en milisegundos que le tomó a MongoDB seleccionar el plan de ejecución y completar la consulta. Es la métrica clave para medir la velocidad de respuesta.
- **totalDocsExamined:** Muestra la cantidad total de documentos que el motor de la base de datos tuvo que revisar en memoria o disco para devolver el resultado final. En una consulta óptima con índices, este número debería aproximarse o ser igual a la cantidad de documentos devueltos.

c) Según la tabla de problemas detectados, si se observa un uso alto de memoria debido a documentos muy grandes, ¿cuál es la solución recomendada? La solución recomendada pasa por dos enfoques principales: la optimización del modelado de datos mediante la separación de datos secundarios en colecciones vinculadas (referenciación en lugar de embebido extremo), y

## 2) Implementation Práctica

```
db.envios.find({ numero_seguimiento: "TRK-003" }).explain("executionStats")
```

Al revisar la salida del comando anterior en la sección `executionStages`, se identifica que el campo `stage` tiene el valor `IXSCAN` (Index Scan), seguido de un stage `FETCH` para recuperar el documento completo de la memoria. Esto verifica que sí se utilizó con éxito el índice creado para el campo `numero_seguimiento` en la Actividad 1.

```
// c) Ejecutar el comando stats() sobre la colección y reportar el tamaño total de los índices
db.envios.stats()
// Nota: En versiones nuevas de mongosh se suele usar db.envios.stats().indexSizes o:
db.envios.dataSize()
db.envios.totalIndexSize() // Este reporta el tamaño exacto solicitado en bytes.
```

## ACTIVIDAD 3: Procesamiento Avanzado con Agregaciones

### 1) Respuestas Teóricas

a) ¿Cuál es la función del Pipeline de Agregación y qué significa que funcione como una "tubería"?

La función del Pipeline de Agregación es procesar y transformar conjuntos de documentos para devolver resultados calculados o combinados. Funciona como una "tubería" porque los documentos pasan secuencialmente por una serie de etapas (stages); la salida o resultado de una etapa se convierte automáticamente en la entrada de la siguiente, permitiendo filtrar, agrupar y transformar los datos de forma progresiva.

b) Describir brevemente para qué sirven las etapas \$match, \$group y \$project.

- **\$match:** Actúa como un filtro. Selecciona únicamente los documentos que cumplen con las condiciones especificadas, reduciendo el volumen de datos que pasan a las siguientes etapas.
- **\$group:** Agrupa los documentos según una clave o criterio definido (\_id) y permite aplicar funciones de agregación (como sumas, promedios o conteos) sobre los grupos generados.
- **\$project:** Modifica la estructura de los documentos de salida. Sirve para renombrar, agregar, calcular o suprimir campos, determinando qué datos se mostrarán en el resultado final.

c) ¿Qué operador de agregación utilizarías para unir la colección envios con una colección de clientes?

Utilizaría el operador \$lookup, que realiza una operación equivalente a un Left Outer Join de SQL para traer documentos de otra colección basándose en campos comunes (clave primaria y foránea).

### 2) Implementación Práctica

Corriendo esto en la terminal

```
db.envios.aggregate([
  { $match: { estado: "Entregado" } },
  { $group: { _id: "$ciudad_destino", total_entregados: { $sum: 1 } } }
])

db.envios.aggregate([
  { $group: { _id: null, peso_total_acumulado: { $sum: "$peso" } } }
])

db.envios.aggregate([
```

```

{
  $project: {
    _id: 0,
    numero_seguimiento: { $toUpper: "$numero_seguimiento" },
    estado: 1
  }
}
})

db.envios.aggregate([
{
  $addFields: {
    tipo_carga: {
      $cond: { if: { $gt: [ "$peso", 20 ] }, then: "Pesada", else: "Ligera" }
    }
  }
}
])

```

Me da este resultado

```

> MONGODB
< {
  _id: ObjectId('6a3a9dc388148ffa262cb6'),
  numero_seguimiento: 'TRK-001',
  estado: 'En Camino',
  ciudad_destino: 'Buenos Aires',
  peso: 5.5,
  tipo_carga: 'Ligera'
}
{
  _id: ObjectId('6a3a9dc388148ffa262cb7'),
  numero_seguimiento: 'TRK-002',
  estado: 'Entregado',
  ciudad_destino: 'Rosario',
  peso: 25,
  tipo_carga: 'Pesada'
}
{
  _id: ObjectId('6a3a9dc388148ffa262cb8'),
  numero_seguimiento: 'TRK-003',
  estado: 'Entregado',
  ciudad_destino: 'Buenos Aires',
  peso: 12.2,
  tipo_carga: 'Ligera'
}
{
  _id: ObjectId('6a3a9dc388148ffa262cb9'),
  numero_seguimiento: 'TRK-004',
  estado: 'Retenido',
  ciudad_destino: 'Córdoba',
  peso: 42.1,
  tipo_carga: 'Pesada'
}
{
  _id: ObjectId('6a3a9dc388148ffa262cba'),
  numero_seguimiento: 'TRK-005',
  estado: 'En Camino',
  ciudad_destino: 'Mendoza',
  peso: 8,
  tipo_carga: 'Ligera'
}
{
  _id: ObjectId('6a3a9dc388148ffa262cbb'),
  numero_seguimiento: 'TRK-006',
  estado: 'Entregado',
  ciudad_destino: 'Rosario',
  peso: 3.2,

```